LeetCode 核心题型知识点教程（由浅入深）

> 本教程基于 `key_point.md` 考点清单，涵盖 8 大核心题型：双指针、滑动窗口、二分法、二叉树、DFS/BFS、自定义排序、优先队列、STL 灵活运用。
>
> 每个章节包含：基本概念 → 核心模板 → 例题精讲（由易到难）→ 常见陷阱与技巧。

目录

- 一、双指针
- 二、滑动窗口
- 三、二分法
- 四、二叉树
- 五、DFS / BFS
- 六、自定义排序
- 七、优先队列
- 八、STL 灵活运用

一、双指针

1.1 基本概念

双指针是指在遍历数据结构时使用两个指针协同工作，通过巧妙地移动指针来减少时间复杂度。常见模式有三类：

模式	典型场景	时间复杂度
快慢指针（同向）	原地修改数组、去重、移除元素	O(n)
对撞指针（相向）	有序数组两数之和、回文验证	O(n)
排序 + 双指针	三数之和、最接近的三数之和	$O(n^2)$

1.2 核心模板

模板一：快慢指针（原地覆盖）

```
// 适用于：移动零、删除重复元素等"原地修改"场景
int slow = 0;
for (int fast = 0; fast < n; fast++) {
    if (nums[fast] 满足保留条件) {
        nums[slow++] = nums[fast];
    }
}
// slow 即为新数组的有效长度
```

模板二：对撞指针

```
// 适用于：有序数组中找两数关系
int left = 0, right = n - 1;
while (left < right) {
    if (满足条件) {
        return 结果;
    } else if (需要左指针右移) {
        left++;
    } else {
        right--;
    }
}
```

模板三：排序 + 双指针（三数之和模式）

```
// 固定一个数，剩余部分用对撞双指针
sort(nums.begin(), nums.end());
for (int i = 0; i < n - 2; i++) {
    if (i > 0 && nums[i] == nums[i - 1]) continue; // 去重
    int left = i + 1, right = n - 1;
    while (left < right) {
        int sum = nums[i] + nums[left] + nums[right];
        if (sum == target) { /* 记录结果 */ left++; right--; }
        else if (sum < target) left++;
        else right--;
    }
}
```

1.3 例题精讲

【简单】283. 移动零 — 快慢指针

题意：将数组中所有 `0` 移到末尾，保持非零元素相对顺序。

思路：用快慢指针，`fast` 遍历全数组，遇到非零就复制到 `slow` 位置，最后将尾部填 `0`。

```cpp
void moveZeroes(vector<int>& nums) {
    int slow = 0;
    for (int fast = 0; fast < nums.size(); fast++) {
        if (nums[fast] != 0) {
            nums[slow++] = nums[fast];
        }
    }
    while (slow < nums.size()) {
        nums[slow++] = 0;
    }
}
```

【简单】26. 删除有序数组中的重复项 — 快慢指针

思路：有序数组中重复元素相邻，`slow` 指向当前唯一元素末尾，`fast` 找到不等的就前移。

```cpp
int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;
    int slow = 1;
    for (int fast = 1; fast < nums.size(); fast++) {
        if (nums[fast] != nums[fast - 1]) {
            nums[slow++] = nums[fast];
        }
    }
    return slow;
}
```

【中等】11. 盛最多水的容器 — 对撞指针

题意：两条竖线与 x 轴围成的容器，求最大面积。

思路：面积 = `min(h[left], h[right]) * (right - left)`。从两端向中间逼近，每次移动较短的一边（因为移动长边不可能增大面积）。

```cpp
int maxArea(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int ans = 0;
    while (left < right) {
        int area = min(height[left], height[right]) * (right - left);
        ans = max(ans, area);
        if (height[left] < height[right]) left++;
        else right--;
    }
    return ans;
}
```

【中等】15. 三数之和 — 排序 + 双指针

题意：找出数组中所有和为 0 的不重复三元组。

思路：排序后固定 `nums[i]`，用左右指针在剩余部分找 `nums[left] + nums[right] = -nums[i]`。注意去重。

```
vector<vector<int>> threeSum(vector<int>& nums) {
    vector<vector<int>> ans;
    sort(nums.begin(), nums.end());
    int n = nums.size();
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // 去重
        int left = i + 1, right = n - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                ans.push_back({nums[i], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return ans;
}
```

【中等】16. 最接近的三数之和 — 双指针逼近

思路：与三数之和类似，但每一步记录与 target 的差值，更新最接近的结果。

1.4 常见陷阱与技巧

1. 去重逻辑：三数之和中，`i` 的去重用 `nums[i] == nums[i-1]`（跳过与前一个相同的），而不是 `nums[i] == nums[i+1]`（后者会漏解）。
2. 对撞指针移动方向：盛水容器中必须移动短边，移动长边面积只会更小。
3. 排序的前提：对撞双指针要求数组有序；三数之和模式先排序再操作。
4. 时间复杂度：双指针通常将 $O(n^2)$ 优化到 $O(n)$，排序+双指针从 $O(n^3)$ 优化到 $O(n^2)$。

二、滑动窗口

2.1 基本概念

滑动窗口是一种特殊的双指针技术，维护一个满足约束的窗口，通过右边界扩展、左边界收缩来高效求解子串/子数组问题。

核心两类：

- 定长窗口：窗口大小固定，右边界每移动一步，左边界同步移动。

- 变长窗口：窗口大小可变，右边界不断扩展，当窗口不满足条件时收缩左边界。

2.2 核心模板

模板一：变长窗口（求最长/最短）

```
// 求满足条件的【最长】子串/子数组
int left = 0, ans = 0;
unordered_map<char, int> cnt; // 窗口内计数
for (int right = 0; right < n; right++) {
    // 1. 扩展右边界：将 nums[right] 加入窗口
    cnt[s[right]]++;
    // 2. 收缩左边界：当窗口不满足条件时
    while (窗口不满足条件) {
        cnt[s[left]]--;
        left++;
    }
    // 3. 更新答案（此时窗口满足条件）
    ans = max(ans, right - left + 1);
}
```

```
// 求满足条件的【最短】子串/子数组
int left = 0, ans = INT_MAX;
for (int right = 0; right < n; right++) {
    // 1. 扩展右边界
    // 2. 当窗口满足条件时，收缩左边界求最短
    while (窗口满足条件) {
        ans = min(ans, right - left + 1);
        // 移出 s[left]，left++
    }
}
```

模板二：定长窗口

```
// 窗口大小固定为 k
for (int i = 0; i < n; i++) {
    // 加入窗口
    cnt[s[i]]++;
    if (i >= k) {
        // 移出最左元素
        cnt[s[i - k]]--;
    }
    if (i >= k - 1) {
        // 窗口 [i-k+1, i] 满足大小 k，处理结果
    }
}
```

2.3 例题精讲

【中等】3. 无重复字符的最长子串 — 变长窗口求最长

题意：找不含重复字符的最长子串长度。

思路：右指针扩展加入字符，当某字符计数 > 1 时收缩左边界直到无重复。

```
int lengthOfLongestSubstring(string s) {
    int left = 0, ans = 0;
    unordered_map<char, int> cnt;
    for (int right = 0; right < s.size(); right++) {
        cnt[s[right]]++;
        while (cnt[s[right]] > 1) {
            cnt[s[left]]--;
            left++;
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

【中等】209. 长度最小的子数组 — 变长窗口求最短

题意：找和 ≥ target 的最短连续子数组。

思路：右边界累加，当和 ≥ target 时收缩左边界并更新最短长度。

```
int minSubArrayLen(int target, vector<int>& nums) {
    int left = 0, sum = 0, ans = INT_MAX;
    for (int right = 0; right < nums.size(); right++) {
        sum += nums[right];
        while (sum >= target) {
            ans = min(ans, right - left + 1);
            sum -= nums[left++];
        }
    }
    return ans == INT_MAX ? 0 : ans;
}
```

【中等】904. 水果成篮 — 变长窗口 + 计数

题意：两个篮子各装一种水果，求最多连续摘多少个（窗口内最多 2 种）。

思路：窗口内维护字符计数和种类数，超过 2 种时收缩左边界。

```cpp
int totalFruit(vector<int>& fruits) {
    int left = 0, ans = 0;
    unordered_map<int, int> cnt;
    for (int right = 0; right < fruits.size(); right++) {
        cnt[fruits[right]]++;
        while (cnt.size() > 2) {
            cnt[fruits[left]]--;
            if (cnt[fruits[left]] == 0) cnt.erase(fruits[left]);
            left++;
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

【中等】438. 找到字符串中所有字母异位词 — 定长窗口

思路：窗口大小 = p 的长度，用计数数组比较窗口和 p 的字符频率。

【中等】1004. 最大连续 1 的个数 III — 窗口 + 允许翻转 K 次

思路：窗口内允许最多 K 个 0，当 0 的个数超过 K 时收缩左边界。

2.4 常见陷阱与技巧

1. 何时用 while 收缩：变长窗口中，收缩条件取决于求最长（不满足时收缩）还是求最短（满足时收缩）。
2. 数据结构选择：字符频率用 `int[26]` 或 `int[128]` 数组比 `unordered_map` 更快；种类数可以用额外变量维护。
3. 窗口合法性判断：先扩展再收缩，最后更新答案——顺序不能错。
4. 定长 vs 变长：定长窗口不需要 while 收缩，只需在 `i >= k` 时移出最左元素。

三、二分法

3.1 基本概念

二分查找通过每次将搜索区间折半，在有序或具有二段性的数据中高效定位目标。核心要求：

- 数据具有单调性或二段性（能根据条件将区间分为两半）
- 每次都能确定目标在左半还是右半

时间复杂度：O(log n)

3.2 核心模板

二分法的难点在于边界处理。以下三个模板覆盖绝大多数场景：

模板一：标准二分（精确查找）

```
int binarySearch(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

模板二：找左边界（第一个 ≥ target）

```
int lowerBound(vector<int>& nums, int target) {
    int left = 0, right = nums.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < target) left = mid + 1;
        else right = mid; // mid 可能是答案，不跳过
    }
    return left; // 第一个 >= target 的位置
}
```

模板三：找右边界（第一个 > target）

```
int upperBound(vector<int>& nums, int target) {
    int left = 0, right = nums.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] <= target) left = mid + 1;
        else right = mid;
    }
    return left; // 第一个 > target 的位置
}
```

3.3 例题精讲

【简单】704. 二分查找 — 标准模板

题意：在有序数组中找目标值。

思路：直接套用模板一。

【简单】35. 搜索插入位置 — 找左边界

题意：找目标值位置，不存在则返回应插入的位置。

思路：等价于找第一个 ≥ target 的位置，用模板二。

```
int searchInsert(vector<int>& nums, int target) {
    int left = 0, right = nums.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < target) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

【中等】34. 查找元素的第一个和最后一个位置 — 左右边界

思路：用两次二分——一次找左边界（第一个等于 target），一次找右边界（最后一个等于 target）。

```
vector<int> searchRange(vector<int>& nums, int target) {
    int first = lowerBound(nums, target);
    if (first == nums.size() || nums[first] != target) return {-1, -1};
    int last = upperBound(nums, target) - 1;
    return {first, last};
}
```

【中等】33. 搜索旋转排序数组 — 旋转数组二分

思路：旋转数组在 mid 处至少有一半是有序的。判断 target 在有序的那一半还是另一半。

```
int search(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) { // 左半有序
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // 右半有序
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

【中等】162. 寻找峰值 — 二分逼近局部极值

思路：比较 `nums[mid]` 和 `nums[mid+1]`，往上升方向走，必定能找到峰值。

3.4 常见陷阱与技巧

1. 防溢出：`mid = left + (right - left) / 2`，不要写 `(left + right) / 2`。
2. 循环条件：`left <= right`（闭区间）vs `left < right`（左闭右开），取决于 right 的初始值和更新方式。
3. 二段性是关键：旋转数组虽不是全局有序，但具有二段性——一半一定有序，同样可以二分。
4. 找边界而非精确值：很多题目不是找等于 target 的位置，而是找第一个满足/不满足某条件的位置，用 `left < right` 模板更安全。

四、二叉树

4.1 基本概念

二叉树问题的核心思维模式：

模式	说明	典型应用
DFS 递归（前序/中序/后序）	通过递归函数遍历或处理	深度、翻转、路径
BFS 层序（队列）	逐层处理	层序遍历、最短路径
递归分治	将问题分解为子树问题	平衡判断、直径

二叉树节点定义：

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

4.2 核心模板

模板一：DFS 递归（后序为例）

```
void postorder(TreeNode* root) {
    if (!root) return;
    postorder(root->left);   // 左
    postorder(root->right);  // 右
    // 处理当前节点（根）
}
```

模板二：BFS 层序遍历

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> ans;
    if (!root) return ans;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int size = q.size();
        vector<int> level;
        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        ans.push_back(level);
    }
    return ans;
}
```

模板三：迭代中序遍历（栈）

```
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> ans;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) {
            st.push(cur);
            cur = cur->left;
        }
        cur = st.top(); st.pop();
        ans.push_back(cur->val);
        cur = cur->right;
    }
    return ans;
}
```

4.3 例题精讲

【简单】104. 二叉树的最大深度 — DFS 递归

```
int maxDepth(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

【简单】226. 翻转二叉树 — 递归交换

```
TreeNode* invertTree(TreeNode* root) {
    if (!root) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```

【简单】101. 对称二叉树 — 递归对比

思路：比较左子树和右子树是否镜像对称。

```
bool isSymmetric(TreeNode* root) {
    return check(root->left, root->right);
}
bool check(TreeNode* a, TreeNode* b) {
    if (!a && !b) return true;
    if (!a || !b) return false;
    return a->val == b->val
        && check(a->left, b->right)
        && check(a->right, b->left);
}
```

【简单】94. 二叉树的中序遍历 — 迭代（栈）

思路：用栈模拟递归，不断往左走到底，弹出后转向右子树。

【中等】102. 二叉树的层序遍历 — BFS 队列

思路：使用队列逐层处理，每层先记录 `size` 再依次出队。

4.4 常见陷阱与技巧

1. 递归基线条件：始终先判断 `root == nullptr`，避免空指针。
2. 前/中/后序的选择：
   - 前序（根左右）：适合"自顶向下"传递信息（如路径前缀和）。
   - 后序（左右根）：适合"自底向上"汇总信息（如深度、平衡判断）。
   - 中序（左根右）：BST 中序遍历得到有序序列。
3. BFS 层序的 size 固定技巧：每层开始前记录 `queue.size()`，这样即使入队了新节点也只处理当前层。
4. 递归返回值：想清楚函数返回什么——是深度？是布尔？还是节点指针？

五、DFS / BFS

5.1 基本概念

DFS（深度优先搜索）和 BFS（广度优先搜索）是图/网格遍历的两大核心方法：

特性	DFS	BFS
数据结构	栈（递归或显式栈）	队列
遍历顺序	一条路走到底再回头	逐层扩展
适用场景	连通性、路径搜索	最短路径、层序
空间复杂度	O(递归深度)	O(最宽层)

网格问题常用方向数组：

```
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};
```

5.2 核心模板

模板一：网格 DFS（染色/连通块）

```
void dfs(vector<vector<char>>& grid, int i, int j) {
    int m = grid.size(), n = grid[0].size();
    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] != '1') return;
    grid[i][j] = '0'; // 标记已访问（染色）
    dfs(grid, i - 1, j);
    dfs(grid, i + 1, j);
    dfs(grid, i, j - 1);
    dfs(grid, i, j + 1);
}
```

模板二：网格 BFS（多源最短距离）

```
vector<vector<int>> bfs(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    vector<vector<int>> dist(m, vector<int>(n, -1));
    queue<pair<int,int>> q;
    // 多源入队
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            if (grid[i][j] == 2) { // 腐烂橘子
                q.push({i, j});
                dist[i][j] = 0;
            }
    // BFS 扩展
    int dx[] = {-1, 1, 0, 0}, dy[] = {0, 0, -1, 1};
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx >= 0 && nx < m && ny >= 0 && ny < n
                && grid[nx][ny] == 1 && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
    return dist;
}
```

模板三：DFS 回溯（路径搜索）

```
void backtrack(参数) {
    if (终止条件) {
        记录结果;
        return;
    }
    for (选择 : 选择列表) {
        做选择;
        backtrack(新参数);
        撤销选择; // 回溯
    }
}
```

5.3 例题精讲

【中等】200. 岛屿数量 — DFS 染色

题意：统计网格中 '1' 连通块的数量。

思路：遍历每个格子，遇到 '1' 就 DFS 染色为 '0'（沉岛），计数 +1。

```cpp
int numIslands(vector<vector<char>>& grid) {
    int m = grid.size(), n = grid[0].size(), ans = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                ans++;
            }
        }
    }
    return ans;
}
```

【中等】994. 腐烂的橘子 — 多源 BFS

题意：所有腐烂橘子每分钟传染四邻，求全部腐烂的最短时间。

思路：所有腐烂橘子同时入队（多源 BFS），逐层扩展，最后一层的距离就是答案。

【中等】79. 单词搜索 — 网格 DFS 回溯

思路：每个格子作为起点 DFS 匹配单词，匹配失败就回溯（恢复访问状态）。

```cpp
bool exist(vector<vector<char>>& board, string word) {
    int m = board.size(), n = board[0].size();
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            if (dfs(board, word, i, j, 0)) return true;
    return false;
}
bool dfs(vector<vector<char>>& board, string& word, int i, int j, int k) {
    if (k == word.size()) return true;
    if (i < 0 || i >= board.size() || j < 0 || j >= board[0].size()
        || board[i][j] != word[k]) return false;
    char tmp = board[i][j];
    board[i][j] = '#'; // 标记
    bool found = dfs(board, word, i-1, j, k+1) || dfs(board, word, i+1, j, k+1)
              || dfs(board, word, i, j-1, k+1) || dfs(board, word, i, j+1, k+1);
    board[i][j] = tmp; // 回溯
    return found;
}
```

【中等】207. 课程表 — BFS 拓扑排序

思路：建图 + 入度表，BFS 不断移除入度为 0 的节点，最后判断是否全部处理完。

【中等】130. 被围绕的区域 — 从边界 DFS/BFS

思路：从矩阵四条边的 'O' 出发 DFS/BFS 标记为不被围绕的 'O'，最后未被标记的 'O' 改为 'X'。

5.4 常见陷阱与技巧

1. 访问标记的方式：
    - 原地修改（染色）：把网格值改为特殊字符，省空间但不保留原数据。
    - 额外 `visited` 数组：不改原数据，但需要 O(mn) 额外空间。
2. 回溯必须恢复状态：单词搜索中 DFS 完要恢复 `board[i][j]`，否则影响其他路径。
3. 多源 BFS：腐烂橘子问题中所有源同时入队，相当于超级源点连到每个腐烂橘子。
4. BFS 天然求最短路径：无权图中 BFS 第一次到达终点的步数就是最短距离。
5. 方向数组技巧：用 `dx/dy` 数组代替四个 if，代码更简洁。

六、自定义排序

6.1 基本概念

自定义排序是指根据题目需求定义特殊的比较函数，将数据按自定义规则排序后再用贪心等策略求解。C++ 中常用 `sort` 配合自定义比较器或 lambda。

```
// sort 默认升序
sort(nums.begin(), nums.end());

// 自定义比较器（lambda）
sort(nums.begin(), nums.end(), [](const int& a, const int& b) {
    return a > b; // 降序
});
```

6.2 核心模板

模板一：自定义比较函数

```
// 方式一：lambda
sort(items.begin(), items.end(), [](auto& a, auto& b) {
    return 自定义条件; // 返回 true 表示 a 排在 b 前面
});

// 方式二：重载 operator<
struct Item {
    int val;
    bool operator<(const Item& other) const {
        return val < other.val;
    }
};
```

模板二：多关键字排序

```
sort(logs.begin(), logs.end(), [](auto& a, auto& b) {
    // 第一关键字
    if (第一关键字不同) return 条件;
    // 第二关键字
    return 第二关键字条件;
});
```

6.3 例题精讲

【中等】179. 最大数 — 拼接字符串自定义比较

题意：将非负整数数组排列成最大的数字字符串。

思路：转为字符串，比较 `a+b` 和 `b+a` 谁更大来决定顺序。

```
string largestNumber(vector<int>& nums) {
    vector<string> strs;
    for (int n : nums) strs.push_back(to_string(n));
    sort(strs.begin(), strs.end(), [](string& a, string& b) {
        return a + b > b + a; // 拼接后比较
    });
    string ans;
    for (string& s : strs) ans += s;
    // 处理前导零
    return ans[0] == '0' ? "0" : ans;
}
```

【中等】451. 根据字符出现频率排序 — 按频率排序

思路：统计频率，按频率降序拼接字符串。

【简单】937. 重新排列日志文件 — 多关键字自定义排序

思路：字母日志在前（按内容排序，内容相同按标识符排序），数字日志保持原序。

【中等】56. 合并区间 — 按左端点排序 + 扫描

思路：按区间左端点排序，遍历时如果当前区间左端 ≤ 上一区间右端就合并。

```cpp
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> ans;
    for (auto& iv : intervals) {
        if (!ans.empty() && iv[0] <= ans.back()[1]) {
            ans.back()[1] = max(ans.back()[1], iv[1]); // 合并
        } else {
            ans.push_back(iv);
        }
    }
    return ans;
}
```

【中等】452. 用最少数量的箭引爆气球 — 按右端点排序贪心

思路：按右端点排序，贪心地从最左端射箭，一箭贯穿尽可能多的气球。

```cpp
int findMinArrowShots(vector<vector<int>>& points) {
    sort(points.begin(), points.end(), [](auto& a, auto& b) {
        return a[1] < b[1];
    });
    int arrows = 1;
    int end = points[0][1];
    for (int i = 1; i < points.size(); i++) {
        if (points[i][0] > end) { // 当前气球在射箭点右侧
            arrows++;
            end = points[i][1];
        }
    }
    return arrows;
}
```

6.4 常见陷阱与技巧

1. 比较函数的严格弱序：比较器必须满足严格弱序——不能出现 `a < b` 且 `b < a` 同时成立（即不能有"等于"的情况用 `<` 判断），否则 `sort` 行为未定义。
2. 拼接比较的精髓：`a+b > b+a` 而非直接比较字符串大小，因为 `"9"` > `"30"` 但 `"930"` < `"309"` 是不对的，实际 `"9"+"30" = "930"` > `"30"+"9" = "309"`。
3. 排序方向的选择：
   - 合并区间：按左端点升序，便于顺序扫描合并。
   - 射箭问题：按右端点升序，贪心选最早结束的。
4. 稳定性：C++ `sort` 不保证稳定，需要稳定排序时用 `stable_sort`。

七、优先队列

7.1 基本概念

优先队列（堆）是一种能 O(log n) 插入、O(1) 取最值、O(log n) 删除最值的数据结构。核心应用场景：

- TopK 问题：维护大小为 K 的堆，流式处理数据。
- 合并 K 个有序序列：每次取最小元素。
- 贪心策略：每次选最优（如 Huffman 编码、重构字符串）。

C++ 中的优先队列：

```
// 大顶堆（默认）
priority_queue<int> maxHeap;

// 小顶堆
priority_queue<int, vector<int>, greater<int>> minHeap;

// 自定义比较
auto cmp = [](pair<int,int>& a, pair<int,int>& b) {
    return a.second > b.second; // 小顶堆（按 second 排序）
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
```

7.2 核心模板

模板一：小顶堆维护 TopK（最大的 K 个）

```
// 求最大的 K 个元素：用小顶堆，堆顶是 K 个中最小的
priority_queue<int, vector<int>, greater<int>> minHeap;
for (int num : nums) {
    minHeap.push(num);
    if (minHeap.size() > k) minHeap.pop(); // 踢出最小的
}
// 堆中剩余的就是最大的 K 个
```

模板二：大顶堆维护 TopK（最小的 K 个）

```
// 求最小的 K 个元素：用大顶堆，堆顶是 K 个中最大的
priority_queue<int> maxHeap;
for (int num : nums) {
    maxHeap.push(num);
    if (maxHeap.size() > k) maxHeap.pop(); // 踢出最大的
}
```

7.3 例题精讲

【中等】347. 前 K 个高频元素 — 小顶堆维护 TopK

题意：返回频率最高的 K 个元素。

思路：先统计频率，再用小顶堆按频率维护大小为 K 的堆。

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> freq;
    for (int n : nums) freq[n]++;

    auto cmp = [](pair<int,int>& a, pair<int,int>& b) {
        return a.second > b.second; // 小顶堆按频率
    };
    priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);

    for (auto& [num, count] : freq) {
        pq.push({num, count});
        if (pq.size() > k) pq.pop();
    }

    vector<int> ans;
    while (!pq.empty()) {
        ans.push_back(pq.top().first);
        pq.pop();
    }
    return ans;
}
```

【中等】215. 数组中的第 K 个最大元素 — 堆 / 快速选择

思路一：小顶堆维护 K 个元素，堆顶就是第 K 大。

思路二（快速选择）：类似快排的 partition，平均 O(n)。

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int n : nums) {
        pq.push(n);
        if (pq.size() > k) pq.pop();
    }
    return pq.top();
}
```

【中等】973. 最接近原点的 K 个点 — 大顶堆维护 TopK

思路：计算每个点到原点距离，用大顶堆维护 K 个距离最小的点。

【中等】264. 丑数 II — 小顶堆生成序列

思路：每次从堆中取最小丑数，乘以 2/3/5 后入堆，用集合去重。

```cpp
int nthUglyNumber(int n) {
    priority_queue<long, vector<long>, greater<long>> pq;
    unordered_set<long> seen;
    pq.push(1);
    seen.insert(1);
    int factors[] = {2, 3, 5};
    long ugly = 1;
    for (int i = 0; i < n; i++) {
        ugly = pq.top(); pq.pop();
        for (int f : factors) {
            long next = ugly * f;
            if (!seen.count(next)) {
                seen.insert(next);
                pq.push(next);
            }
        }
    }
    return ugly;
}
```

【中等】767. 重构字符串 — 大顶堆贪心

思路：按字符频率建大顶堆，每次取频率最高的两个字符拼接（避免相邻相同），频率减 1 后重新入堆。

7.4 常见陷阱与技巧

1. 堆的方向选择：
   - 求最大的 K 个 → 用小顶堆（堆顶是 K 个中最小的，便于踢出）。
   - 求最小的 K 个 → 用大顶堆（堆顶是 K 个中最大的，便于踢出）。
   - 口诀：找大的用小堆，找小的用大堆。
2. 去重：丑数问题中同一丑数可能通过不同路径产生，必须用集合去重。
3. 自定义比较器的写法：`priority_queue` 的比较器是"返回 true 时 a 的优先级低于 b"（与 `sort` 相反），容易混淆。
4. 快速选择 vs 堆：第 K 大问题中，快速选择平均 O(n) 更快但最坏 $O(n^2)$；堆始终 O(n log k) 稳定。

八、STL 灵活运用

8.1 基本概念

C++ STL 提供了丰富的容器和算法，灵活组合可以高效解决很多问题。本节聚焦以下容器：

容器	核心特性	典型应用
`stack`	LIFO 后进先出	括号匹配、单调栈
`unordered_map`	O(1) 平均查找	计数、分组、缓存
`list`	双向链表，O(1) splice	LRU 缓存
单调 `stack`	维护单调性	下一个更大元素

8.2 核心模板

模板一：栈 — 括号匹配

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty()) return false;
            char top = st.top(); st.pop();
            if (!匹配(c, top)) return false;
        }
    }
    return st.empty(); // 栈必须为空
}
```

模板二：单调栈 — 下一个更大元素

```
vector<int> nextGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n, -1);
    stack<int> st; // 存索引，栈内元素对应的值单调递减
    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[st.top()] < nums[i]) {
            ans[st.top()] = nums[i]; // 找到下一个更大元素
            st.pop();
        }
        st.push(i);
    }
    return ans;
}
```

模板三：LRU 缓存 — list + unordered_map

```
class LRUCache {
    int cap;
    list<pair<int,int>> lru; // {key, value}, 前端是最近使用的
    unordered_map<int, list<pair<int,int>>::iterator> mp; // key -> 迭代器

public:
    LRUCache(int capacity) : cap(capacity) {}

    int get(int key) {
        if (!mp.count(key)) return -1;
        lru.splice(lru.begin(), lru, mp[key]); // 移到前端
        return mp[key]->second;
    }

    void put(int key, int value) {
        if (mp.count(key)) {
            mp[key]->second = value;
            lru.splice(lru.begin(), lru, mp[key]);
        } else {
            if (lru.size() == cap) {
                mp.erase(lru.back().first);
                lru.pop_back();
            }
            lru.push_front({key, value});
            mp[key] = lru.begin();
        }
    }
};
```

8.3 例题精讲

【简单】20. 有效的括号 — stack 配对

思路：左括号入栈，右括号弹出栈顶匹配。最后栈必须为空。

```
bool isValid(string s) {
    stack<char> st;
    unordered_map<char,char> match = {{')','('}, {']','['}, {'}','{'}};
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty() || st.top() != match[c]) return false;
            st.pop();
        }
    }
    return st.empty();
}
```

【中等】155. 最小栈 — 辅助 stack

思路：用一个额外的栈同步记录当前最小值。每次 push 时也 push 当前最小值。

```cpp
class MinStack {
    stack<int> st, minSt;
public:
    void push(int val) {
        st.push(val);
        minSt.push(minSt.empty() ? val : min(minSt.top(), val));
    }
    void pop() { st.pop(); minSt.pop(); }
    int top() { return st.top(); }
    int getMin() { return minSt.top(); }
};
```

【中等】739. 每日温度 — 单调 stack

题意：对每一天，找下一个更暖和的日子距几天。

思路：单调递减栈存索引，遇到更高温度时弹出并计算距离。

```cpp
vector<int> dailyTemperatures(vector<int>& temps) {
    int n = temps.size();
    vector<int> ans(n, 0);
    stack<int> st; // 单调递减栈，存索引
    for (int i = 0; i < n; i++) {
        while (!st.empty() && temps[st.top()] < temps[i]) {
            ans[st.top()] = i - st.top();
            st.pop();
        }
        st.push(i);
    }
    return ans;
}
```

【中等】49. 字母异位词分组 — unordered_map + 排序/计数

思路：将每个字符串排序（或计数字母）作为 key，相同 key 的分到一组。

```cpp
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> mp;
    for (string& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        mp[key].push_back(s);
    }
    vector<vector<string>> ans;
    for (auto& [k, v] : mp) ans.push_back(v);
    return ans;
}
```

【中等】146. LRU 缓存 — list + unordered_map（splice）

思路：`list` 维护访问顺序（前端最近使用），`unordered_map` 做 key 到 list 节点的 O(1) 定位，`splice` 实现 O(1) 节点移动。

8.4 常见陷阱与技巧

1. 栈的空判检查：弹出前必须检查 `!st.empty()`，否则 `top()` / `pop()` 是未定义行为。
2. 单调栈的方向：
   - 单调递减栈（栈底到栈顶递减）：找下一个更大元素。
   - 单调递增栈（栈底到栈顶递增）：找下一个更小元素。
3. LRU 的 splice 技巧：`list.splice(dest_pos, source_list, iterator)` 可以 O(1) 将节点从一个位置移到另一个位置，无需拷贝。
4. unordered_map vs map：
   - `unordered_map`：O(1) 平均查找，无序，适合纯查找场景。
   - `map`：O(log n) 查找，有序，需要按 key 遍历时使用。
5. 辅助栈的同步操作：最小栈中 push/pop 必须两个栈同步操作，否则状态不一致。

附录：题型速查表

题型	核心数据结构	核心思想	典型时间复杂度
双指针	两个指针变量	协同移动减少遍历	O(n)
滑动窗口	双指针 + 哈希/计数	维护满足约束的窗口	O(n)
二分法	左右指针	折半搜索	O(log n)
二叉树	递归 / 栈 / 队列	DFS 分治 / BFS 层序	O(n)
DFS/BFS	递归栈 / 队列	连通性 / 最短路径	O(mn)
自定义排序	sort + 比较器	排序后贪心	O(n log n)
优先队列	priority_queue	维护 TopK / 贪心取最值	O(n log k)
STL 运用	stack/map/list	容器组合解题	视场景

附录：练习进度跟踪

题型	题目数	已完成	待完成
双指针	5	1 (283)	4
滑动窗口	5	1 (3)	4
二分法	5	1 (704)	4
二叉树	5	1 (94)	4
DFS/BFS	5	1 (200)	4
自定义排序	5	1 (179)	4
优先队列	5	1 (347)	4
STL 灵活运用	5	1 (20)	4
合计	40	8	32

本教程基于 `key_point.md` 考点清单整理，代码使用 C++17 STL，所有模板均经过实战验证。